Descripción del código y estrategias de paralelización

Proyecto Apertura: DBSCAN paralelo para detectar ruido con OpenMP

Tania Hernandez y Tamara Schnaas

Cómputo Paralelo y en la Nube, ITAM. Entrega: 20 de octubre de 2026

Contents

Versión serial: `src/serial/dbscan_serial.cpp` 2
Versión paralela P1 (matriz indivisible): `src/parallel/dbscan_p1.cpp` 3
Versión paralela P2 (dividida en k): `src/parallel-2/dbscan_p2.cpp` 5
Diferencias clave entre P1 y P2 9
Preguntas que nos pueden hacer en la defensa 9

Las tres versiones implementan el **mismo** algoritmo ingenuo de 2 pasos (diapositivas p5–p10, nota 03), leen el mismo CSV, escriben el mismo CSV y miden el tiempo igual. Solo cambia **cómo se reparte el trabajo**. Por eso el speedup es una comparación justa y las etiquetas deben salir idénticas.

Versión	Archivo	Argumentos
Serial	`src/serial/dbscan_serial.cpp`	`entrada.csv salida.csv eps min_samples`
P1	`src/parallel/dbscan_p1.cpp`	… `num_hilos`
P2	`src/parallel-2/dbscan_p2.cpp`	… `num_hilos k`

Validación: con 4,000, 20,000 y 80,000 puntos (`eps = 0.03`, `min_samples = 10`), P1 con hilos {1, 8, 16, 32} y P2 con los mismos hilos y k ∈ {2, 4, 8, 16, 32, 64} dan **0 etiquetas distintas** respecto a la serial. A su vez, la serial da 0 diferencias respecto al DBSCAN de sklearn (59, 45 y 34 puntos de ruido, respectivamente). Además, el script del experimento volvió a comparar las etiquetas contra la serial en las **224 configuraciones paralelas** (n de 20k a 200k), con 0 diferencias.

Cómo se compila (Mac con Intel y libomp de Homebrew)

Apple clang no acepta `-fopenmp` directo (nota 02, "Cómo compilar en tu Mac"):

```
clang++ -std=c++17 -O2 -Xpreprocessor -fopenmp -I/usr/local/opt/libomp/include \
    -L/usr/local/opt/libomp/lib -lomp dbscan_p1.cpp -o dbscan_p1.out
./dbscan_p1.out 4000_data.csv 4000_results.csv 0.03 10 16
```

Versión serial: `src/serial/dbscan_serial.cpp`

Idea en una frase

Para cada punto se recorren **todos** los puntos: primero para contar vecinos (Paso 1) y luego, solo para el ruido, para buscar un core cercano (Paso 2).

Recorrido del código, bloque por bloque

Líneas	Bloque	Qué hace y por qué
`40-50`	`empiezaConNumero`	Dice si una línea empieza con dígito, signo o punto decimal. `leerPuntos` la usa para **saltarse las líneas que no son datos**: líneas vacías y un encabezado como `x,y`. El CSV de `DBSCAN_noise.ipynb` no trae encabezado, pero uno guardado con pandas sí, y sin esto `stod("x")` lanzaría una excepción y el programa terminaría. Usa `unsigned char` porque `isdigit` con un `char` negativo (acentos en UTF-8) tiene comportamiento indefinido. **No viene de clase**: lo agregamos al probar con otros formatos (`notebooks/prueba_dataset.ipynb`).
`55-95`	`leerPuntos`	Lee el CSV en **dos pasadas**: la primera cuenta las líneas de datos para reservar `x` y `y` del tamaño exacto con `new double[n]` (arreglos dinámicos, nota 02 parte A, p3–p13); la segunda parsea `x,y` separando por la coma. Las dos pasadas usan el mismo filtro (`empiezaConNumero`), así que cuentan y leen exactamente las mismas líneas.
`98-102`	`distancia`	Distancia euclidiana de la p5: `sqrt(dx² + dy²)`. Es `inline` para que el compilador la pegue dentro del ciclo y no pague una llamada a función miles de millones de veces.
`106-115`	Argumentos	`argv` → ruta de entrada, de salida, `eps` (`atof`) y `min_samples` (`atoi`) (p14 de OpenMP).
`123-126`	`es_core`	Un `bool` por punto: `true` = core (de 1er o 2º orden), `false` = ruido. El proyecto solo pide ruido o no ruido, así que no hace falta un id de cluster.
`134`	Cronómetro	`omp_get_wtime()` **después** de leer el archivo: solo se mide el algoritmo, que es lo que se paraleliza. Se usa el mismo reloj en las tres versiones.
`136-154`	Paso 1	Para cada `i`, cuenta los `j` con `distancia <= eps`, **incluido él mismo** (d(i,i) = 0), igual que `min_samples` en sklearn. Si `vecinos >= min_samples`, es core de 1er orden. Son **n^2 distancias**.
`156-191`	Paso 2	Para cada punto que es ruido, busca un core de 1er orden a $\leq$ eps; si lo encuentra, se reetiqueta y hace `break`. Lee de la **copia fija** `es_core_paso1` y escribe en `es_core` (ver abajo).
`196-208`	Salida	`x,y,label` con `label` 1 o 0, el formato que lee la celda 2 del notebook (p14).

Líneas	Bloque	Qué hace y por qué
`206-217`	Reporte	Imprime en `stderr` el tiempo y cuántos puntos quedaron como ruido; el script del experimento lee el tiempo de ahí.
`219-222`	Liberar memoria	`delete[]` de cada arreglo creado con `new[]` (nota 02 parte A).

Decisión clave: la copia `es_core_paso1`

La p8 dice que un ruido se reetiqueta si es épsilon-alcanzable desde **un punto core**, entendido como un core de **primer** orden. Si el Paso 2 leyera y escribiera el mismo arreglo, un punto recién reetiquetado podría "rescatar" a otro revisado después. Eso propagaría en cadena y el resultado **dependería del orden del `for`**. Con la copia fija, el resultado no depende del orden. Esto coincide con el *border point* de sklearn (0 diferencias) y es justo lo que después permite paralelizar el Paso 2 sin condiciones de carrera.

Alternativa considerada: comparar `dist²` `<= eps²` (sin `sqrt`)

Es equivalente matemáticamente: `sqrt` es creciente y ambos lados son ≥ 0. La medimos con 80,000 puntos, promediando 3 corridas:

	`sqrt`	`dist²`
Serial	4.83 s	3.00 s
P1, 8 hilos (speedup)	0.69 s (7.0)	0.50 s (6.1)
P1, 16 hilos (speedup)	0.67 s (7.2)	0.41 s (7.4)
Etiquetas (4k, 20k, 80k)	= sklearn	= sklearn

Decisión: se queda `sqrt`. Es la fórmula de la p5 y el **speedup**, que es lo que pide el proyecto, prácticamente no cambia. El cambio tendría que hacerse en las tres versiones, así que el serial también se acelera. `dist²` solo reduce el tiempo absoluto.

Costo

Paso 1: n^2 distancias. Con n = 200,000 son 4×10^{10}. Duplicar n cuadruplica el tiempo. Los arreglos `x`, `y` de 200,000 `double` ocupan ~3.2 MB y casi caben en caché, así que el algoritmo está limitado por **cómputo**, no por memoria. Es lo contrario de la suma de vectores (nota 01 §13, nota 03).

Versión paralela P1 (matriz indivisible): `src/parallel/dbscan_p1.cpp`

Idea en una frase

El mismo código que la serial, pero el `for` de afuera de cada paso se reparte entre los hilos con `#pragma omp parallel for`. Cada punto se sigue comparando contra los **n** puntos (la "matriz indivisible" de la p11).

Qué cambia respecto a la serial

Líneas	Cambio	Por qué
`106-116`	5º argumento `num_hilos`	El experimento varía los hilos: {1, 8, 16, 32} en esta Mac (p14).
`120`	`omp_set_num_threads(num_hilos)`	Fija el tamaño del equipo de las regiones paralelas que siguen (nota 02, p28).
`151`	`#pragma omp parallel for schedule(static)` en el Paso 1	Ver la tabla de decisiones.
`186`	`#pragma omp parallel for schedule(dynamic, 64)` en el Paso 2	Ver la tabla de decisiones.

Todo lo demás (lectura, `distancia`, cronómetro, copia `es_core_paso1`, salida) es idéntico.

Decisiones de paralelización y su justificación

Decisión	Alternativa descartada	Por qué	Fuente
Paralelizar el `for` **de afuera** (`i`)	El de adentro (`j`)	Afuera hay n iteraciones independientes: se crea el equipo de hilos **una sola vez** por paso. Adentro se crearía y destruiría el equipo n veces, y el conteo de `vecinos` necesitaría `reduction`.	nota 02, p47–p49
`parallel for` (con `parallel`)	`#pragma omp for` solo	Sin `parallel` no hay equipo de hilos y lo hace uno solo.	nota 02, p37–p38
`schedule(static)` en el Paso 1	`dynamic`	Todas las iteraciones cuestan lo mismo (siempre recorren n puntos). Los bloques iguales repartidos desde antes dan carga pareja **sin costo de reparto**.	nota 02, p40–p42
`schedule(dynamic, 64)` en el Paso 2	`static`	Las iteraciones cuestan **muy distinto**: un core sale al instante (`continue`) y un ruido aislado recorre los n puntos sin `break`. Con `static`, un hilo con muchos ruidos se queda solo al final (p43).	nota 02, p43–p44
Bloques (chunk) de 64	chunk 1	Pedir trabajo requiere sincronizar. Con chunk 1 se pagaría eso en cada iteración; con 64 se paga n/64 veces y el balance sigue siendo bueno.	nota 02, p44–p46

Pendiente: medir `static` contra `dynamic` en el Paso 2 con n grande, y probar otros chunks (p45 propone 1…1280). El Paso 2 es mucho más barato que el Paso 1, así que el efecto en el tiempo total debería ser chico.

Por qué es correcto (sin condiciones de carrera)

- **Compartidas y solo leídas:** `x`, `y`, `eps`, `min_samples`, `n_points` y, en el Paso 2, `es_core_paso1`. Nadie las modifica dentro de las regiones paralelas, así que leerlas al mismo tiempo es seguro.
- **Compartida y escrita:** `es_core`. El hilo que procesa `i` solo escribe `es_core[i]`, y el `parallel for` le da a cada hilo valores de `i` distintos. Dos hilos nunca escriben la misma posición, así que **no hace falta `critical` ni `atomic`** (p63–p71).
- **Privadas:** `i` (índice del `parallel for`, privado automáticamente), y `vecinos` y `j`, que se declaran **dentro** del `for` (nota 02, "Cláusulas de parallel").
- **Orden entre pasos:** el Paso 2 necesita el resultado **completo** del Paso 1. La **barrera implícita** al final del primer `parallel for` garantiza que ningún hilo empieza el Paso 2 antes de que todos terminen el Paso 1.
- **Paso 2:** se lee de `es_core_paso1` (fijo) y se escribe en `es_core`. Sin la copia, el resultado dependería de qué hilo llegó primero: sería no determinista (nota 01 §12, p166–p171).

Costo y rendimiento

Hace las **mismas n² distancias** que la serial. Solo se reparten entre hilos, así que lo más que se puede esperar es un speedup cercano al número de **cores físicos** (8). Resultado del experimento (promedio de 10 corridas; el análisis completo está en `experimental-eval.md` §4.1):

n = 200,000	Serial	P1, 8 hilos	P1, 16 hilos	P1, 32 hilos
Tiempo (s)	28.09	4.40	4.61	4.66
Speedup	1	**6.38**	6.09	6.03

Pasar de 8 a 16 hilos casi no ayuda: los 16 hilos son 8 cores físicos con *hyper-threading* y el trabajo es puro cómputo, así que los dos hilos de un core compiten por las mismas unidades de punto flotante. Con 32 hilos hay más hilos que cores virtuales y solo se agrega costo de cambiar de hilo.

Versión paralela P2 (dividida en k): `src/parallel-2/dbscan_p2.cpp`

Idea en una frase

Se parte el **plano** en una cuadrícula de k celdas (p11). Cada celda clasifica solo **sus** puntos, comparándolos contra sus puntos más los vecinos cercanos de las celdas de al lado (el **halo**), y las celdas se reparten entre los hilos.

Cómo se divide la matriz

k	Cuadrícula (filas × columnas)
2	1 × 2 (dos mitades, como en la p11)
4	2 × 2 (cuadrantes, como en la p11)

k	Cuadrícula (filas × columnas)
8	2 × 4
16	4 × 4
32	4 × 8
64	8 × 8

- `cuadricula` (`dbscan_p2.cpp:124-128`) elige la cuadrícula más cuadrada posible: entre más cuadradas sean las celdas, menos perímetro tienen y más chico es el halo.
- Se usa el rectángulo que contiene a todos los puntos (`dbscan_p2.cpp:173-180`) y cada punto cae en exactamente **una** celda, que es su partición (`propios`, `dbscan_p2.cpp:186-193`).
- Se divide el **plano**, no el **orden** de los puntos en el archivo. Si se tomaran "los primeros n/k puntos", cada parte seguiría teniendo vecinos regados por todo el plano y habría que comparar contra todos de nuevo.

Cómo se unen los resultados manteniendo consistencia

Esta es la pregunta de la p11.

El problema: un punto pegado a la frontera de su celda tiene vecinos **del otro lado**. Si la celda solo viera sus propios puntos, ese punto perdería vecinos y podría marcarse como ruido por error. En el Paso 2 pasa lo mismo: el core que lo rescataría podría estar en la celda vecina.

La solución: halo, o zona fantasma (`dbscan_p2.cpp:195-227`). Cada celda tiene dos listas:

Lista	Contenido	Para qué se usa
`propios[c]`	Los puntos que pertenecen a la celda	Son los **únicos** a los que la celda les **escribe** etiqueta.
`candidatos[c]`	Los puntos dentro del rectángulo de la celda **agrandado eps por cada lado**	Solo se **leen**: para contar vecinos (Paso 1) o buscar un core (Paso 2).

Por qué es exacto: si `j` es vecino de `i` (distancia ≤ eps), entonces `|x_j - x_i| <= eps` y `|y_j - y_i| <= eps`. Como `i` está dentro de su celda, `j` está dentro de la celda agrandada eps y por lo tanto en `candidatos`. Cada punto ve **exactamente los mismos vecinos** que en la serial. El margen es un poco mayor que eps (`eps * 1.000001`) para que un redondeo de punto flotante en la orilla no deje fuera a un vecino real. Meter un punto de más no cambia nada, porque igual se revisa su distancia exacta.

Por qué unir es trivial: cada punto pertenece a **una sola** celda, así que su etiqueta se escribe **una sola vez**, directo en el arreglo global `es_core` y en su posición original. No hay conflictos que resolver ni nada que "pegar" al final.

Recorrido del código, bloque por bloque

Líneas	Bloque	Qué hace
`134-152`	Argumentos	Igual que P1, más `k`. Rechaza una `k` que no sea potencia de 2.
`170`	Cronómetro	Arranca **antes** de armar las particiones: el tiempo incluye todo el trabajo extra de P2. Si no, la comparación con la serial lo favorecería injustamente.
`172-193`	Particiones	Rectángulo global, tamaño de celda y asignación de cada punto a su celda.
`195-227`	Halos	Para cada celda, en paralelo, recorre los n puntos y guarda los que caen en la celda agrandada. Guarda índices y una **copia contigua** de sus coordenadas.
`229-263`	Paso 1	`parallel for` sobre las **celdas**. Cada punto propio se compara solo contra los candidatos de su celda.
`265-290`	Paso 2	Copia `es_core_paso1` y otro `parallel for` sobre las celdas, que busca un core de 1er orden entre los candidatos.
`295-302`	Salida	Igual que la serial: `es_core` ya está en el orden original del archivo.

Decisiones de paralelización y su justificación

Decisión	Alternativa descartada	Por qué	Fuente
Un `parallel for` sobre las **k celdas**	`parallel for` sobre los puntos	Es lo que pide la p11: "dividir la matriz por k" y resolver cada parte en paralelo.	p11
`schedule(dynamic, 1)` en los tres `for` de celdas	`static`	Las celdas tienen cantidades **muy distintas** de puntos: los 4 blobs no están repartidos parejo y hay celdas casi vacías. Con `dynamic`, el hilo que termina una celda chica toma otra. Chunk 1 porque solo hay $k \leq 64$ iteraciones y cada una es grande, así que el costo de reparto es despreciable.	nota 02, p43–p44
Barrera entre el Paso 1 y el Paso 2	—	El Paso 2 de una celda lee `es_core_paso1` de puntos del **halo**, que pertenecen a **otra** celda. Todas deben terminar el Paso 1 primero, y la barrera implícita del `parallel for` lo garantiza.	nota 02

Decisión	Alternativa descartada	Por qué	Fuente
Copiar las coordenadas de los candidatos a `cand_x`, `cand_y`	Leer `x[candidatos[c][b]]` directo	**No viene de clase; lo medimos.** Con el acceso indirecto, cada lectura salta a una posición distinta de memoria y el compilador no vectoriza el ciclo. Con 80,000 puntos, k = 2 y 1 hilo, P2 tardaba **5.57 s, más que la serial (4.77 s)**. Con la copia contigua bajó a **2.79 s**.	Medido en esta Mac; nota 01 §13 (memoria y caché)

Por qué es correcto (sin condiciones de carrera)

- `propios`, `candidatos`, `cand_x` y `cand_y` se construyen antes de los pasos. Al construir los halos en paralelo, cada hilo hace `push_back` solo en los vectores **de su celda** `c`.
- **Paso 1:** se escribe `es_core[i]` solo para `i` en `propios[c]`. Cada punto es propio de una sola celda y cada celda la procesa un solo hilo, así que dos hilos nunca escriben la misma posición.
- **Paso 2:** igual que en P1, se lee de la copia fija `es_core_paso1`. Un core de 2º orden no puede rescatar a otro punto, ni dentro de su celda ni a través del halo.
- **Privadas:** `c`, `a`, `b`, `i`, `j`, `vecinos`, `mios` y `cx`/`cy`, todas declaradas dentro del `for`.

Costo y rendimiento

Cada punto ya no se compara contra los n puntos, sino contra los de su celda más el halo. El total de distancias baja aproximadamente de n^2 a n^2/k (un poco más, por el halo y porque los puntos no están repartidos parejo). Así que P2 gana por dos razones: **el paralelismo y un algoritmo con menos trabajo**. Por eso puede superar al número de cores. Resultado del experimento con n = 200,000 (speedup contra la serial, promedio de 10 corridas; análisis en `experimental-eval.md` §4.2):

	k = 2	k = 4	k = 8	k = 16	k = 32	k = 64
1 hilo	1.71	2.70	4.17	5.28	6.59	8.23
8 hilos	3.23	5.03	11.26	13.26	33.91	**41.82**

Limitaciones que se ven en la tabla: - **Con k < hilos sobran hilos:** con k = 2 solo hay 2 celdas, así que solo 2 hilos trabajan aunque haya 8. El speedup con 8 hilos (3.23) apenas es el doble del de 1 hilo. - **Desbalance:** el tiempo de un paso es el de la celda **más cargada**. Con k = 16 la mejora sobre k = 8 es chica. **Confirmado contando el trabajo por celda** (`experiments/analizar_celdas.py`): con la cuadrícula de 4 × 4, una celda concentra el **37%** del trabajo, más que con k = 8 (34%). Con k = 32 baja a 15%, y por eso el salto grande es de k = 16 a k = 32.

Diferencias clave entre P1 y P2

	P1	P2
Qué se reparte entre hilos	Los n puntos	Las k celdas
Distancias totales	n^2 (igual que la serial)	$\approx n^2/k$ + halo
Límite del speedup	~cores físicos (8)	Puede superarlo, porque también reduce trabajo
Hilos útiles	Todos	A lo más k
Balance de carga	Bueno (Paso 1 parejo; Paso 2 con `dynamic`)	Depende de cómo caen los blobs en la cuadrícula
Trabajo extra	Ninguno	Armar celdas y halos: $O(k \cdot n)$, despreciable frente a n^2/k
Consistencia	Trivial: un hilo por posición de `es_core`	Halo de ancho eps + un dueño por punto

Preguntas que nos pueden hacer en la defensa

1. **¿Por qué no hay `critical` en P1 si todos los hilos escriben en `es_core`?** Porque cada hilo escribe posiciones distintas (`es_core[i]` con su propia `i`). La carrera solo existe si dos hilos escriben la misma posición, o si uno escribe una posición que otro está leyendo.
2. **¿Por qué el Paso 2 lee `es_core_paso1` y no `es_core`?** Para que solo los core de 1er orden rescaten ruido (p8). Además, en paralelo el resultado dependería de qué hilo llega primero.
3. **¿Qué garantiza que el Paso 2 vea el Paso 1 completo?** La barrera implícita al final del `parallel for` del Paso 1.
4. **¿Por qué `static` en el Paso 1 y `dynamic` en el Paso 2?** El Paso 1 tiene iteraciones de igual costo. El Paso 2 tiene iteraciones muy desiguales por el `continue` y el `break` (p40–p44).
5. **¿Por qué P1 no mejora de 8 a 16 hilos?** La Mac tiene 8 cores físicos. Los otros 8 son *hyper-threading* y comparten las unidades de cálculo, que es justo lo que este algoritmo satura.
6. **En P2, ¿cómo evitan marcar como ruido un punto en la frontera?** Con el halo: cada celda lee los puntos a menos de eps de su borde, aunque sean de otra celda. Cualquier vecino real está ahí.
7. **¿Cómo se unen los resultados de las k partes?** No hay que unir nada. Cada punto tiene una sola celda dueña, que escribe su etiqueta directo en el arreglo global, en su posición original.
8. **¿Por qué P2 puede tener speedup mayor que el número de cores?** Porque no solo reparte el trabajo: también hace menos distancias que la serial, aproximadamente n^2/k.
9. **¿Por qué con k = 2 y 16 hilos P2 es lenta?** Solo hay 2 celdas, o sea 2 iteraciones del `parallel for`, así que 14 hilos se quedan sin trabajo (p48: "no todos los hilos podrían utilizarse").
10. **¿Por qué el cronómetro de P2 empieza antes de armar las celdas?** Porque armar celdas y halos es trabajo que la serial no hace. Dejarlo fuera inflaría el speedup de P2.
11. **¿Por qué copian las coordenadas del halo a otros arreglos?** Para leer memoria en orden y que el compilador vectorice el ciclo. Sin la copia, P2 con k = 2 y 1 hilo era más lenta que la serial.
12. **¿P2 con 1 hilo ya es 8× más rápida que la serial? ¿Cómo, si no hay paralelismo?** Ese 8× viene de hacer menos distancias: contando, P2 con k = 64 hace $n^2/8.1$ distancias. El paralelismo aporta otro ~5× encima, y $8.2 \times 5.1 \approx 42$.

13. **¿Por qué P2 rinde mejor con 8 hilos que con 16?** El tiempo lo marca la celda más pesada. Con más hilos que cores físicos, el hilo de esa celda puede compartir core por Hyper-Threading e ir más lento. Es una hipótesis: no lo medimos directamente.
14. **Entonces, ¿P2 paraleliza mejor que P1?** No. P1 aprovecha mejor los hilos (~6.4× contra ~5.1×). P2 gana en el total porque hace mucho menos trabajo.
15. **¿Qué pasa si les doy un CSV con encabezado o con líneas vacías?** `empiezaConNumero` se salta toda línea que no empiece con un número, en las dos pasadas de `leerPuntos`. Sin eso, `stod("x")` lanza una excepción. Lo probamos en `notebooks/prueba_dataset.ipynb`: dan las mismas etiquetas que el archivo limpio.
16. **¿Cómo saben que funciona con datos que no sean los del experimento?** `prueba_dataset.ipynb` corre las tres versiones sobre 23 datasets distintos (la receta del notebook del curso con otros `n_points`, `eps` y `min_samples`, otras formas, la carita de la portada, coordenadas negativas, puntos repetidos, un solo punto…). Todos dan 0 diferencias contra sklearn y entre versiones.